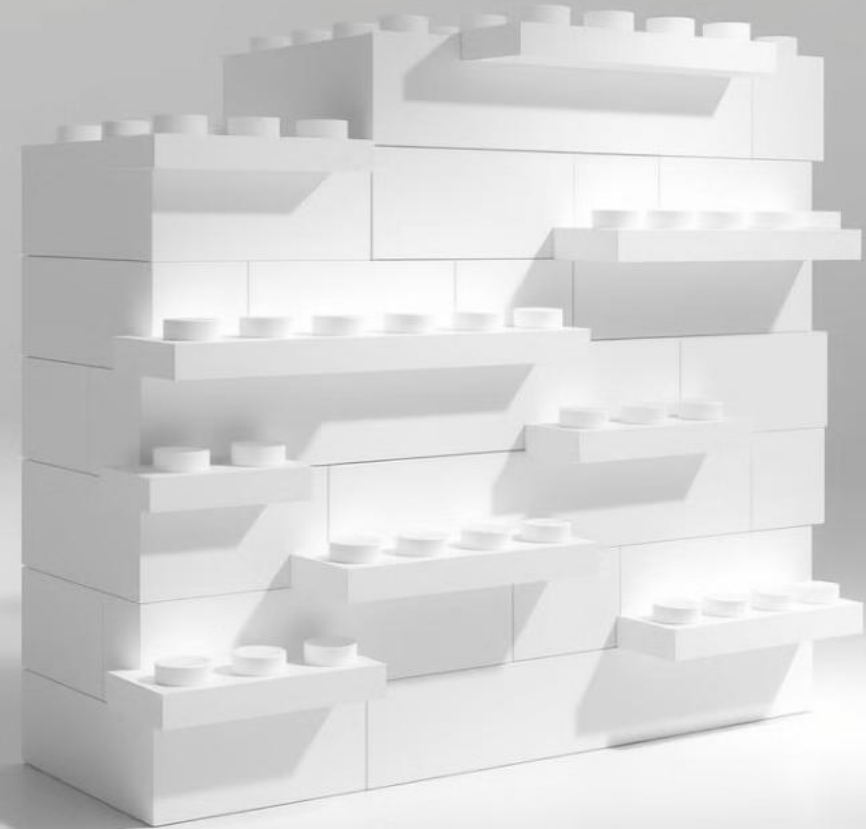


INTRODUCTION TO COMPONENT BASED SOFTWARE DEVELOPMENT

CHAPTER 4 – COMPONENT MODELS WITH OBJECTS AS COMPONENTS



Contents

- 1. POJO's**
- 2. JavaBeans**
- 3. Object Oriented Frameworks**
- 4. COM**
- 5. CCM**
- 6. KorbA**

Component Models with Objects as Components

- In this category of components models
 - components are ***Objects*** defined by Object Oriented Programming languages, and
 - they compose by ***method or event delegation***
- a system built from objects as components is just an Object-Oriented program with a main class

4. CMs with Objects as Components > Major Examples

- the archetypal example of this category is *JavaBeans*
 - here *Java Objects (Beans)* are the components and they are composed by *event delegation*
 - Beans are different from *POJO (Plain Old Java Object)* in that they serializable interface to enable componentization
- *OSGi* uses OOP Objects as fundamental elements but build larger building blocks called *Bundles* which it uses as *base Objects* for composition
 - then to enable component functionality they group these base Objects together
 - mostly used for building large and complex applications in a modular manner
- Multiple *Frameworks* also contain their own Component models built on top of Objects that are composed by delegation

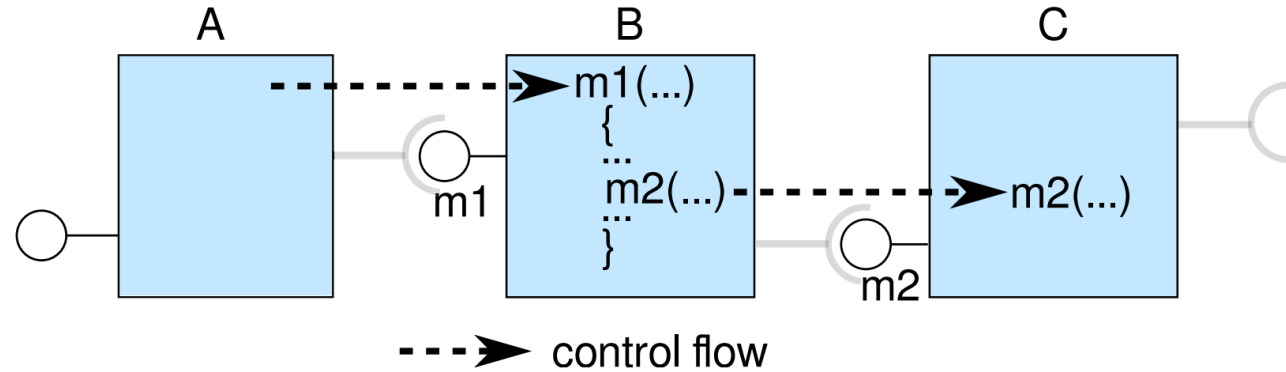
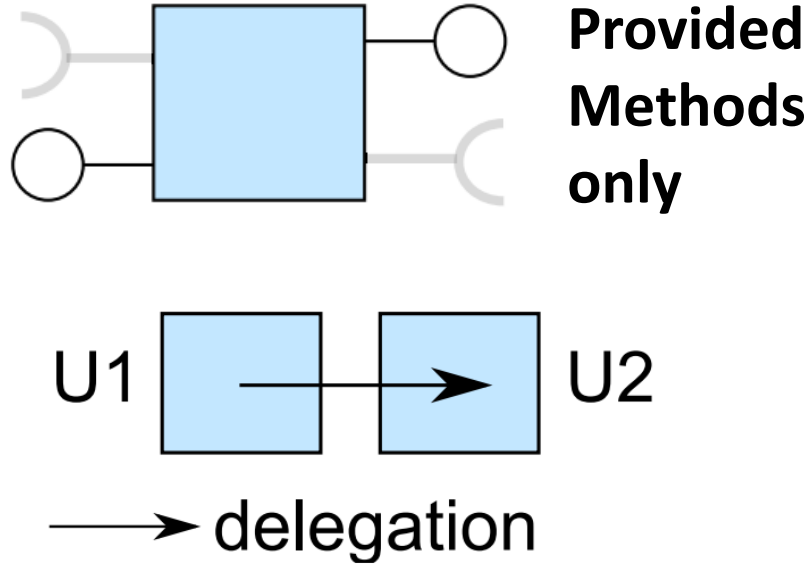
4. CMs with Objects as Components > Major Examples

Non-Algebraic property of Objects as Components

- Composing objects by delegation not an algebraic composition mechanism.
- Composing an object A with another object B does not yield another object rather it yields a pair of objects A,B.
 - $Object \times Object \rightarrow Object \times Object$, - where Object is the type of objects.
- An ***algebraic composition*** operation ***should be*** a function of type
 - $Object \times Object \rightarrow Object$

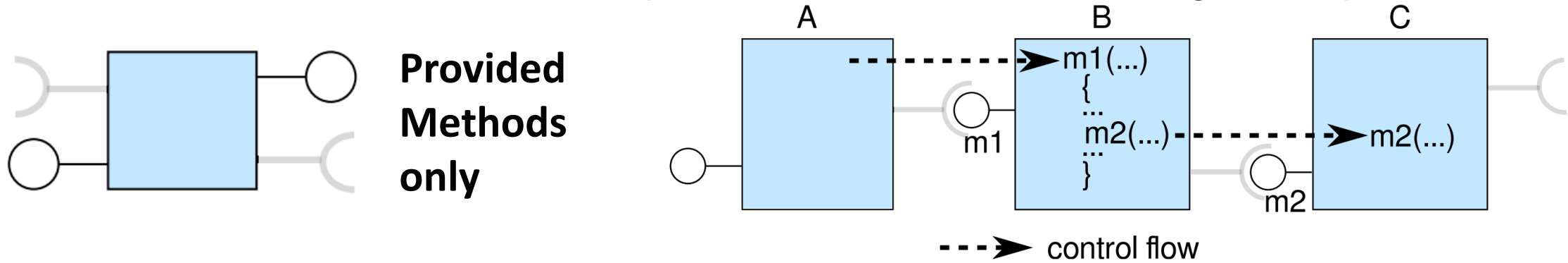
Below we will discuss example Component Models, their component lifecycles with respect to the idealized component lifecycle - to see how well they relate

4.1 POJO's (Plain Old Java Objects)



- if we use sole **POJO's** as components composed by **method or event delegation** then we can consider it component model **fully defined in Java**.
- syntax would be the **Java Class defining syntax**
- semantics and compositions are shown in images above

4.1 POJO's (Plain Old Java Objects)

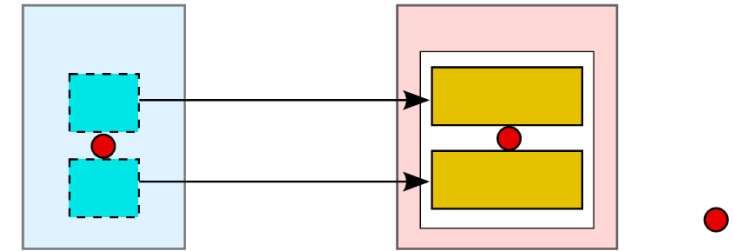


CBD using POJO's would just be Java programming,

- constructing systems requires defining and composing POJO's
 - classes and their method calls
 - including the main class for the system

Relation to Idealized

- Design Phase - use Java IDE to write all POJO's
- Deployment Phase - there is ***no deployment Phase***
- Run-time Phase - Java Virtual Machine (JVM) runs written code
 - only Design phase compositions are compiled



4.2 JavaBeans

```
public class ATM implements Serializable{
    private CardReader reader;
    public ATM(){
        :
    }
    public withdraw(Bank bank, Integer amount){
        bank.withdraw(reader.getAccountNo(), amount);
        :
        this.displayBalance();
    }
    :
    public void setCardReader(CardReader cr){
        reader = cr;
    }
}
```

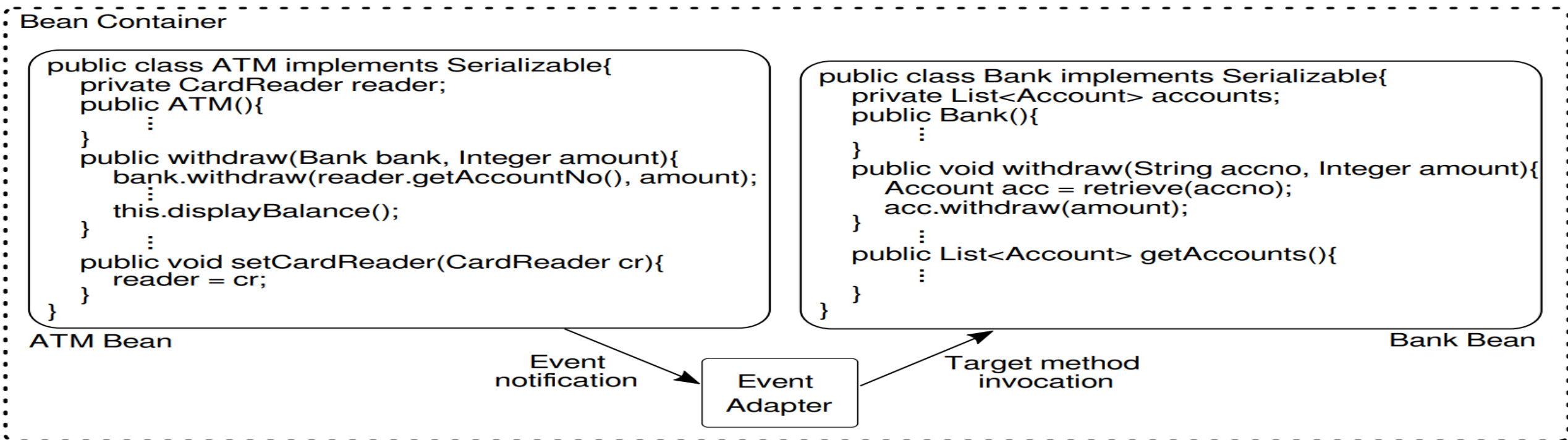
ATM Bean

```
public class Bank implements Serializable{
    private List<Account> accounts;
    public Bank(){
        :
    }
    public void withdraw(String accno, Integer amount){
        Account acc = retrieve(accno);
        acc.withdraw(amount);
    }
    :
    public List<Account> getAccounts(){
        :
    }
}
```

Bank Bean

- a component is a **Bean** (a java object that:)
 - implements **Serializable Interface**
 - 0 argument constructor (**nullary constructor**)
 - private attributes accessible **by getters and setters**
- ex: **Bank & ATM** - ATM - requires Bank.withdraw service

4.2 JavaBeans



- Beans can be constructed in **visual builder** (container for beans)
 - in builder composition is by linking an event in source bean to a method in target bean (**method call**)
 - builder generates **event adapter object** that provides this link (**indirect**)
 - this adaptor object listens for event fire and calls method on target objects

3.2 JavaBeans

- MOST COMMONLY USED FOR - GUI - like buttons, boxes and lists of *java.awt* and *javax.swing packages*

Relation to Idealized

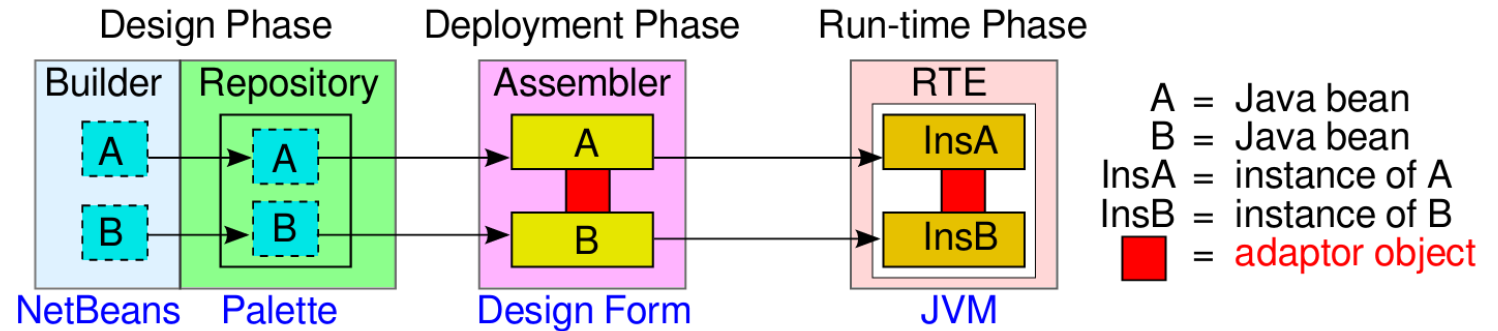
- Design Phase

- Write Java code to build Beans – and manipulate them in Visual Builder
- Then Deposit compiled .JAR files in (Palette-NetBeans) repository

- Deployment Phase

- Use Beans in ***Design Form canvas*** dragged from repository
- Composition between Bean components only at deployment

- Run-time Phase - Java Virtual Machine (JVM) runs written code



4.3 Object-Oriented Frameworks

- these are support providers for object-oriented programming
- Their foundational elements are objects, therefore within the frameworks they contain component models where objects are the components composed by delegation

The three major Object-Oriented Frameworks are

- **Enterprise JavaBeans**
- **OSGi**
- **.NET**

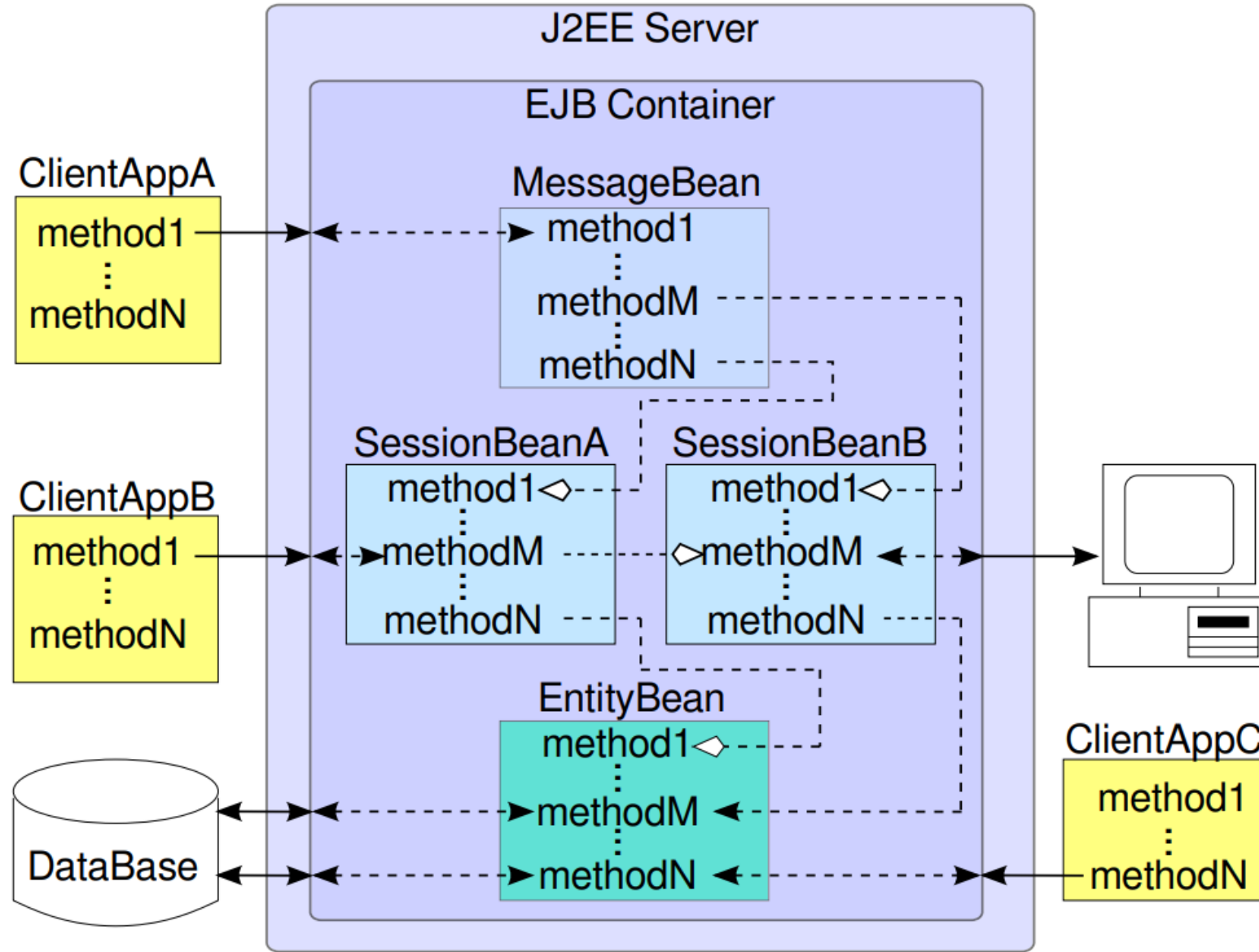
4.3.1 Enterprise JavaBeans

- a framework for a high-level approach to building distributed systems by creating ***server-side components*** that can be ***accessed remotely over network by client applications***

Composition

- EJB components can collaborate with each other to execute remote client application requests via ***event and method delegation***
- These Composition cannot create full systems
 - only clients contain the special ***main classes*** to run programs that use these services

4.3.1 Enterprise JavaBeans (EJBs)



How it works in Deployment

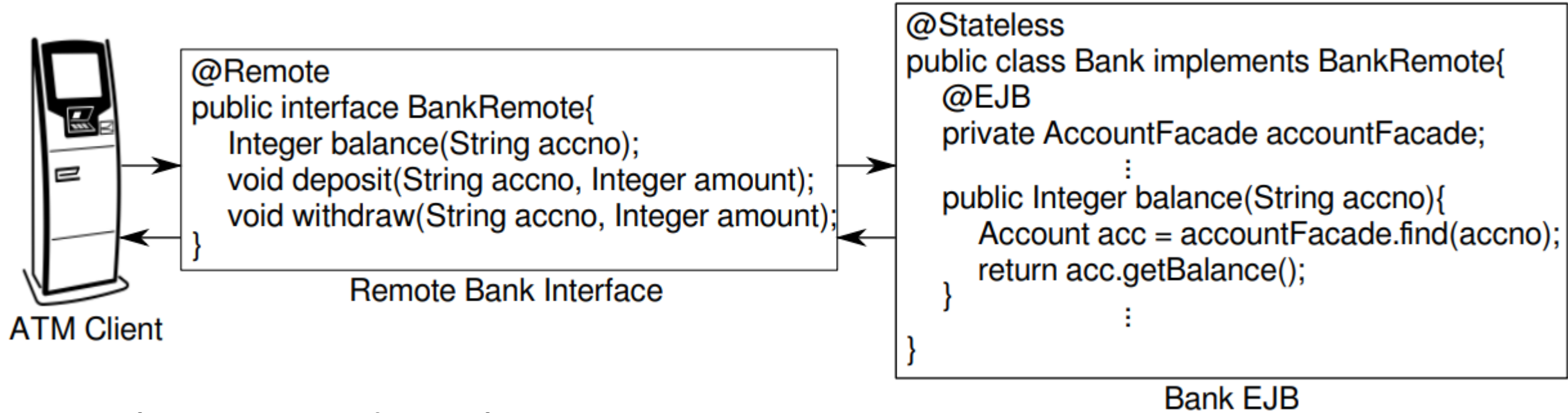
- **J2EE servers** such as *Glassfish* and *JBoss* host and manage **EJB containers**
- **Servers** are execution environments for EJB's (Enterprise Java Beans)
- **EJB's** are these Java Objects that can be accessed remotely from this environment

4.3.1 Enterprise JavaBeans > *3 types of EJBs*

There are 3 types of EJB's

- **Entity Beans:** provides Object-oriented view of a database's data
- **Session Beans:** agents that perform tasks
 - represents work performed by the client - can be *stateless* or *stateful*
 - stateful session beans associate with a specific client to maintain conversational state during session (like cookies and catching)
- **Message-Driven Beans:** message listeners

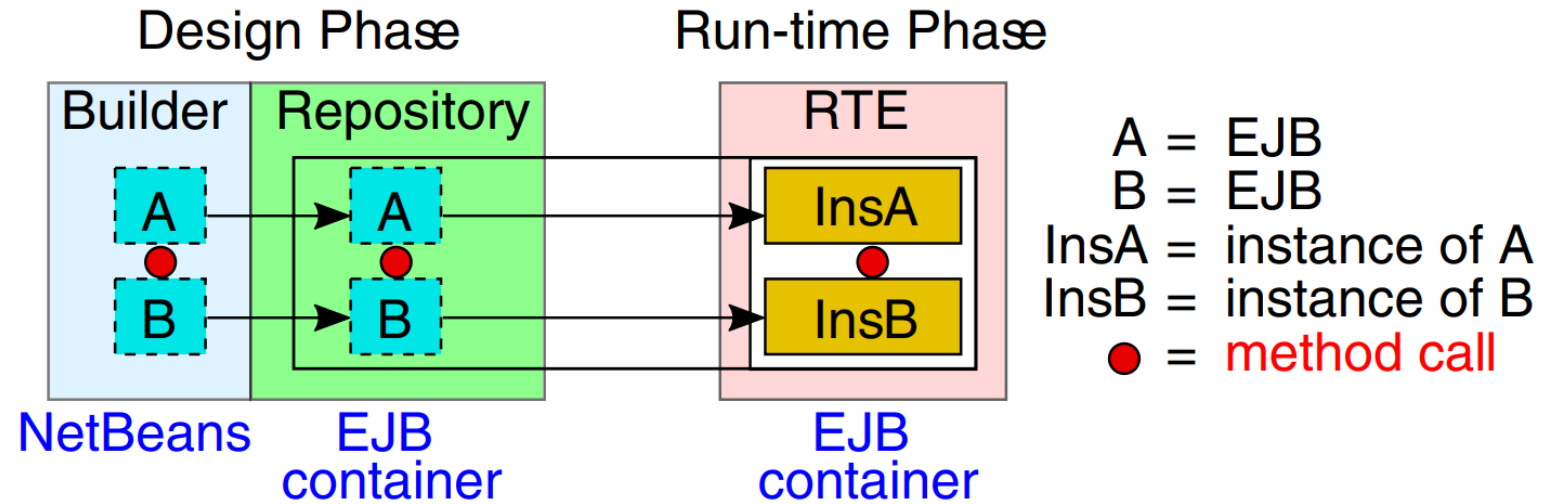
4.3.1 Enterprise JavaBeans > 3 types of EJBs > Example



Example ATM and Bank EJB

- for every EJB object/component - its EJB container generates a ***remote interface*** that ***EJBs implement*** to expose provided services (available services over a network to a remote client)
- in this example we have ***Account Entity Bean***, ***AccountFacade*** helper and a ***stateless Bank SessionBean***

4.3.1 Enterprise JavaBeans > 3 types of EJBs



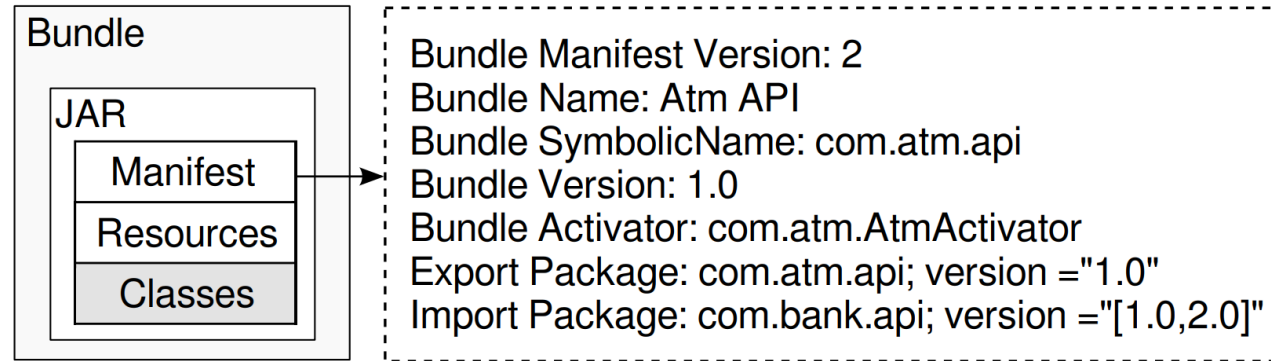
Relation to Idealized

- Design Phase:
 - J2EE compliant IDE (like NetBeans) used to build and compose in Java Beans into .JAR files
 - .JAR files for beans deposited into EJB containers (**the repository**)
- Deployment Phase: there is ***no Deployment Phase***, no further composition from EJB containers (No Visual Builder)
- Runtime Phase: composition remains the same as design phase, EJB's just execute when invoked, no changes!

4.3.2 OSGi (Open Services Gateway Initiative) platform

- a framework that introduces modularity to Java by defining **Bundles**

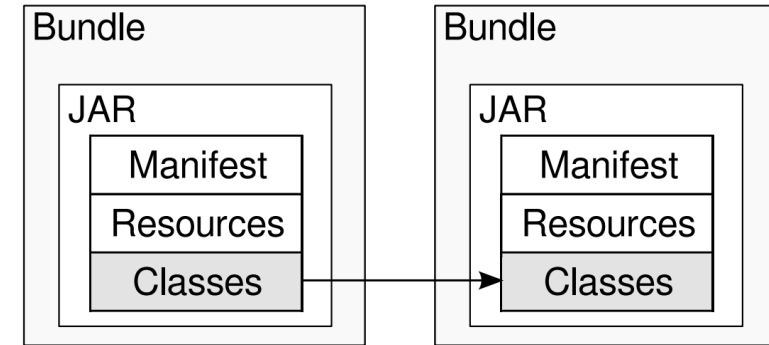
Bundle



- like EJB's bundles contain objects (POJOs) that interact with each other to provide services to client applications.
 - bundles also provide services to one-another
- The compiled bundle .JAR file contains files for classes and resources that define the Bundle,
 - within the JAR we also have .manifest file for description - **name, version, import and export packages**

4.3.2 OSGi (Open Services Gateway Initiative) platform

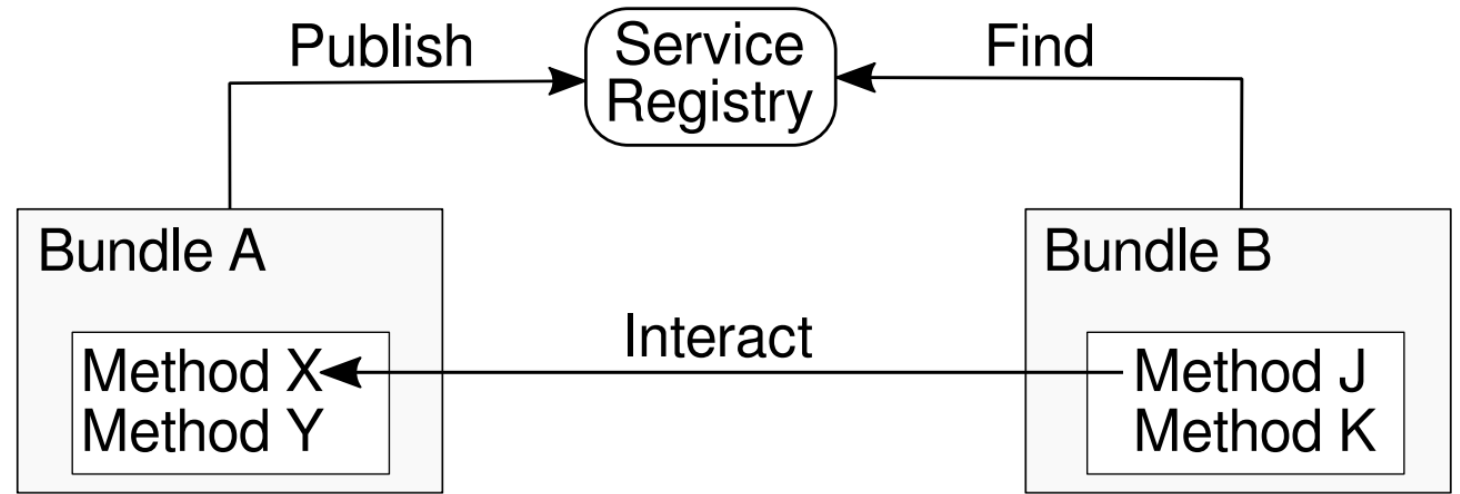
- OSGi compliant frameworks (like Equinox and Felix) serve Bundles just like J2EE servers serve EJB containers



Framework Vs Component Model?

- this ***Bundle*** is called a component in OSGi, but:
 - no composition model is defined – as OSGi is not a composition model!
 - no composite bundles.
- but since classes in bundles can be linked by method call - we say they compose by delegation
 - hence OSGi is a framework that ***contains a component model*** with Objects (POJOs) as components
 - POJOs are linked/composed in design phase by **delegation**

4.3.2 OSGi (Open Services Gateway Initiative) platform

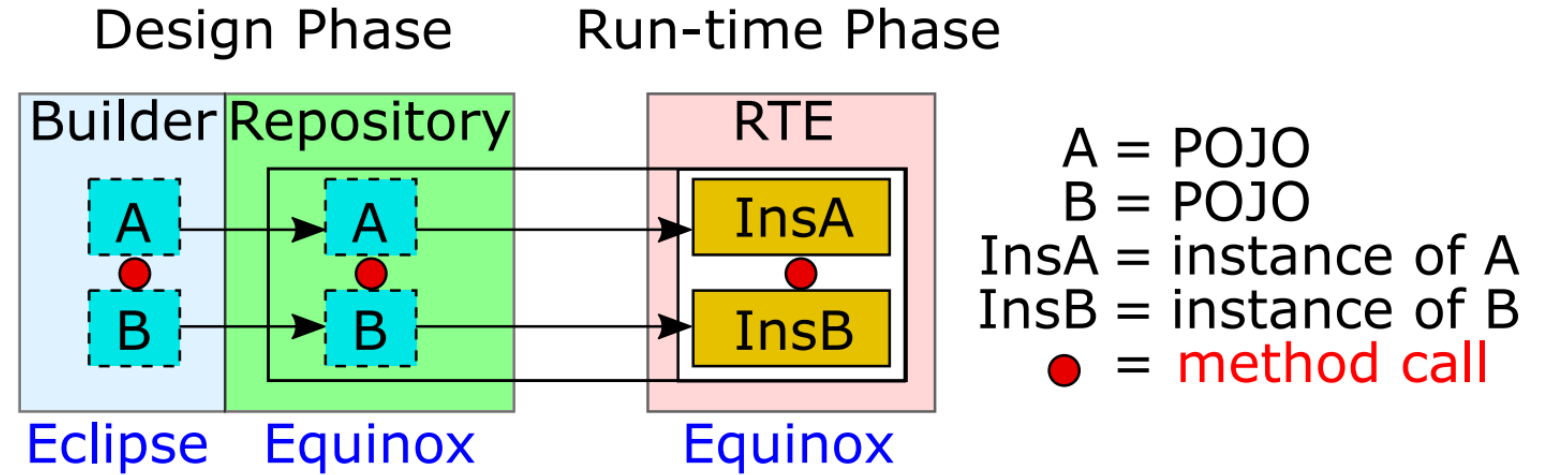


Composition is Realized at Runtime

- method calls *implement only in runtime*
- at Runtime - OSGi implements **service registry** within the **JVM (Java Virtual Machine)**
 - each bundle .manifest analyzed and offered services are registered
 - bundles can look up registry for required services

(for this reason, OSGi is called a Service-Oriented Architecture (SOA))

4.3.2 OSGi (Open Services Gateway Initiative) platform



Relation to Idealized

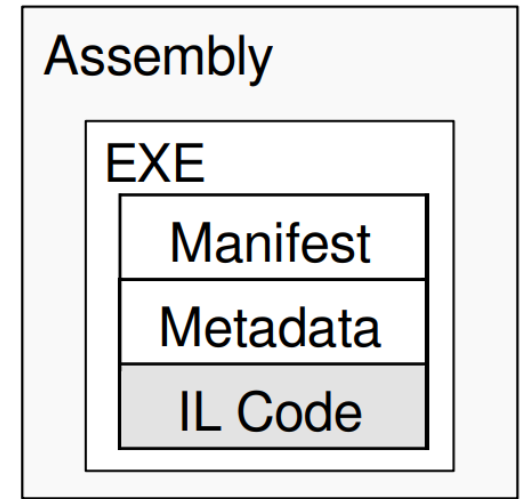
- Design Phase:
 - Bundles built in editor (like Eclipse), and compiled to expose the service
 - Bundles (.JAR) installed in OSGi-compliant framework (like Equinox) - which serves as ***the repository***
- Deployment Phase:- ***no deployment phase***
- Run-time Phase:- client applications use services provided by bundles by calling POJOs within bundles via their published service interface in registry

4.3.3 .NET

- a framework that introduces modularity
 - by defining **Assemblies**

Assemblies

- a binary modular unit with several files, mainly classes.
 - all .NET languages compile into ***IL (Microsoft's platform independent intermediate language)***
 - IL contains: ***DLL (dynamic link libraries)*** and ***(EXE) files*** to execute code in classes
- the IL code is compiled at runtime into machine code by ***JIT (Just-in-Time) compiler***
- an assembly provides services performed by these classes to its client applications
- an **assembly** also contains:
 - a manifest file with - **name, version, id, location & dependencies**
 - a metadata file with - classes, interfaces, methods and attributes

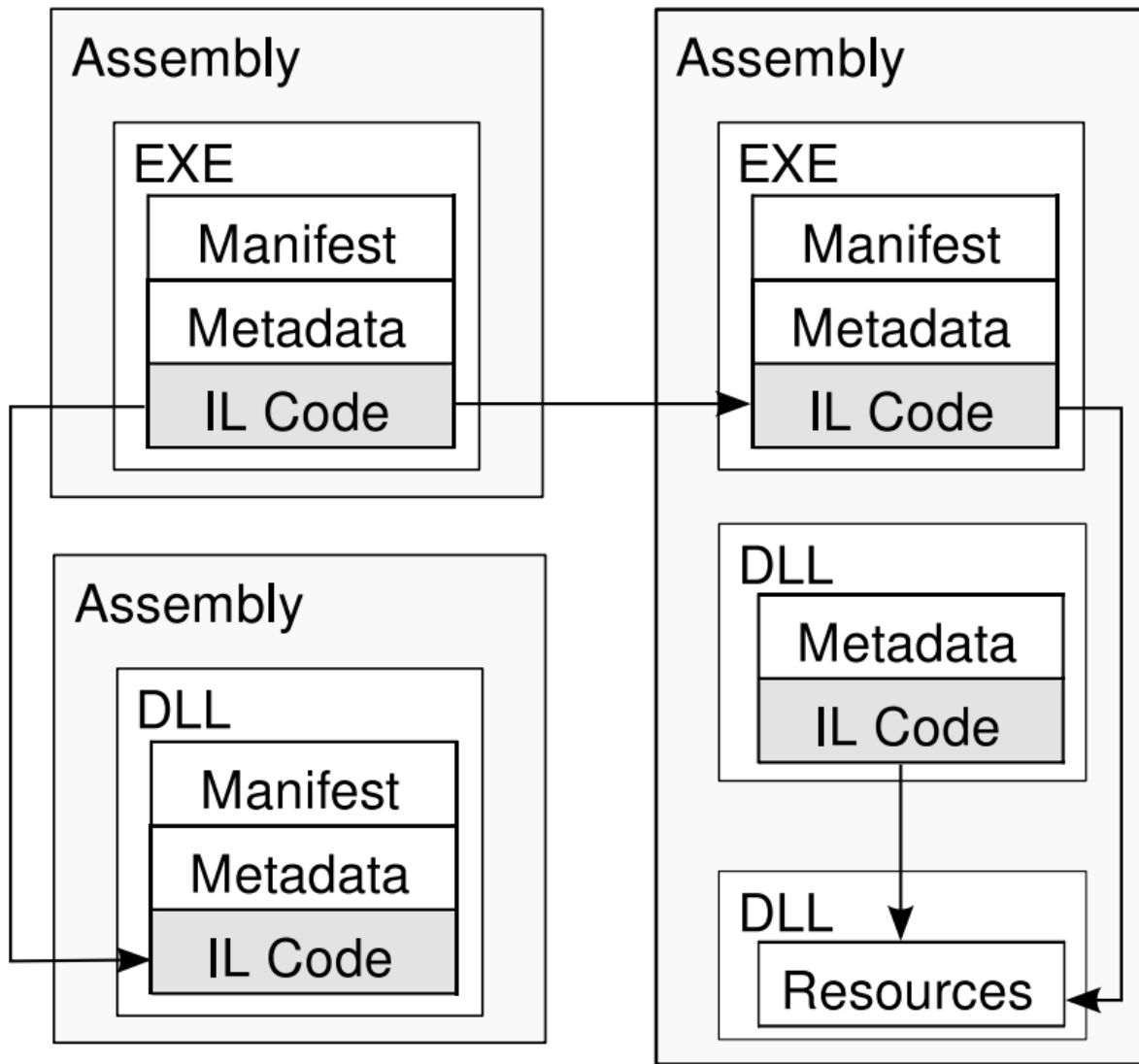


4.3.3 .NET

Framework Vs Component Model?

- An **Assembly** is called a *component* in .NET, but:
 - no composition model defined - .NET is not a composition model!
 - no composite Assemblies exist.
- but since classes in Assemblies can be linked by method call - we say they compose by **delegation**
 - hence .NET is a framework that contains a component model with Objects (POJOs) as components
 - POJOs are linked/composed in design phase by delegation

4.3.3 .NET



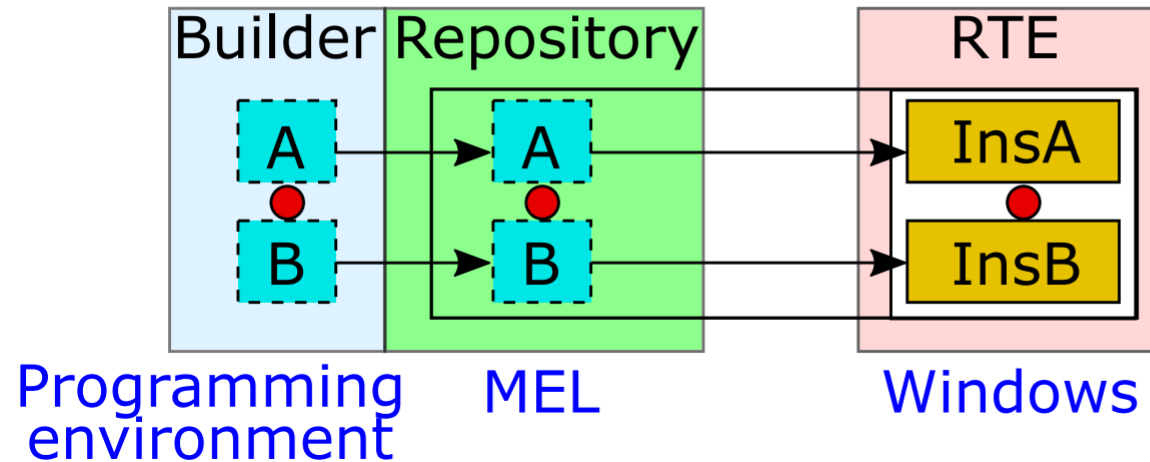
Composition is Realized at Runtime
- *method calls* implementation

- at Runtime - JIT compiler links client calls to required IL entry point (manifest interface)
- when compiling .NET to IL - stubs for class methods are produced in machine code
- at execution JIT compiler gets IL, compiles it, replacing stub in memory with the machine code

4.3.3 .NET

Design Phase

Run-time Phase



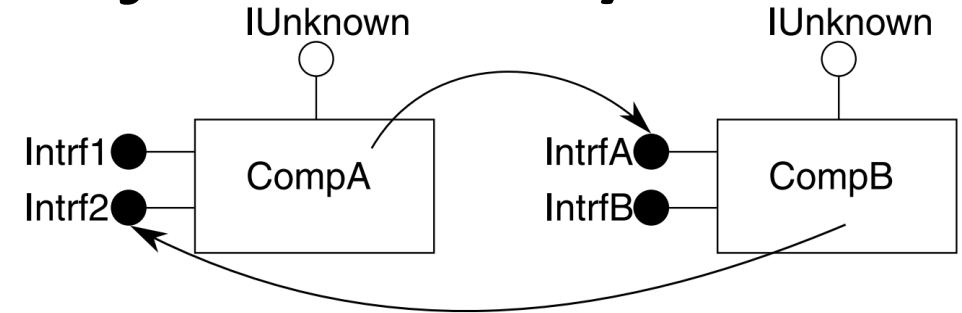
Relation to Idealized

- Design Phase
 - Assemblies designed and built in C# or V.Basic in editor like MS Visual Studio
 - compiled to .Net Assemblies and deposited MS Enterprise Library (MEL)
(deposit only repository)
- - Deployment Phase: no deployment phase - no further composition
- Run-time Phase:
 - client applications use assemblies by invoking .NET classes within them
 - object composition is as it is in design phase

4.4 COM (Component Object Model)

- COM has been superseded by .NET, we see it just for historical interest
- Its components are similar to POJOs and JavaBeans, composed by **delegation**
- It is just like EJB with provided services that can be used by client applications

4.4 COM (Component Object Model)



COM components - don't necessarily have to be Objects (can be defined with any programming language with any internal state), But

- they should provide functionalities that can be called!
 - Also functions of a component don't need to be methods,
- but any executable binary that can be called via **pointers** (to the **interface** of the component)
 - i.e. interface is a contractual way for component to expose its services
- components implement the ***IUnknown Interface***,
 - to control their own lifecycle
 - To dynamically determine other components services/capabilities

4.4 COM (Component Object Model)

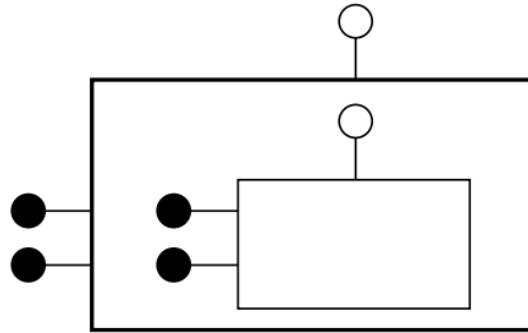
```
import "unknown.idl"
[object, uuid(10000001-0000-0000-0000)]
interface withdraw: IUnkown
{
    HRESULT withdraw([in] Bank bank,
                     [in] Integer amount,
                     [out, retval] int* balance);
};
:
```

For language-neutrality:

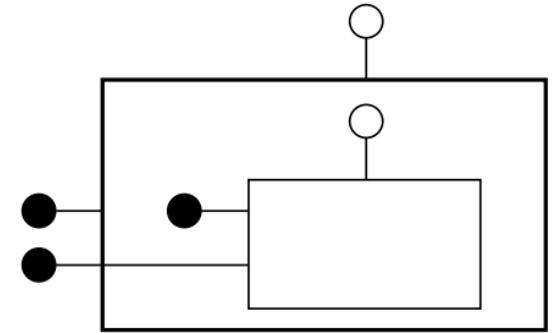
- COM Interfaces are defined in MS Interface definition language (MIDL)
 - each interface has unique ID for name clashing
 - [in]/[out] directional attributes
- MIDL can be compiled into Programming Languages in Visual studio
 - - developer can implement them in components
 - - implementing components are then deposited to ***Windows registry***

4.4 COM (Component Object Model)

- like OSGi - pointers and interfaces are built at design time but actual linking happens a run-time within **COM application server**



(a) Containment

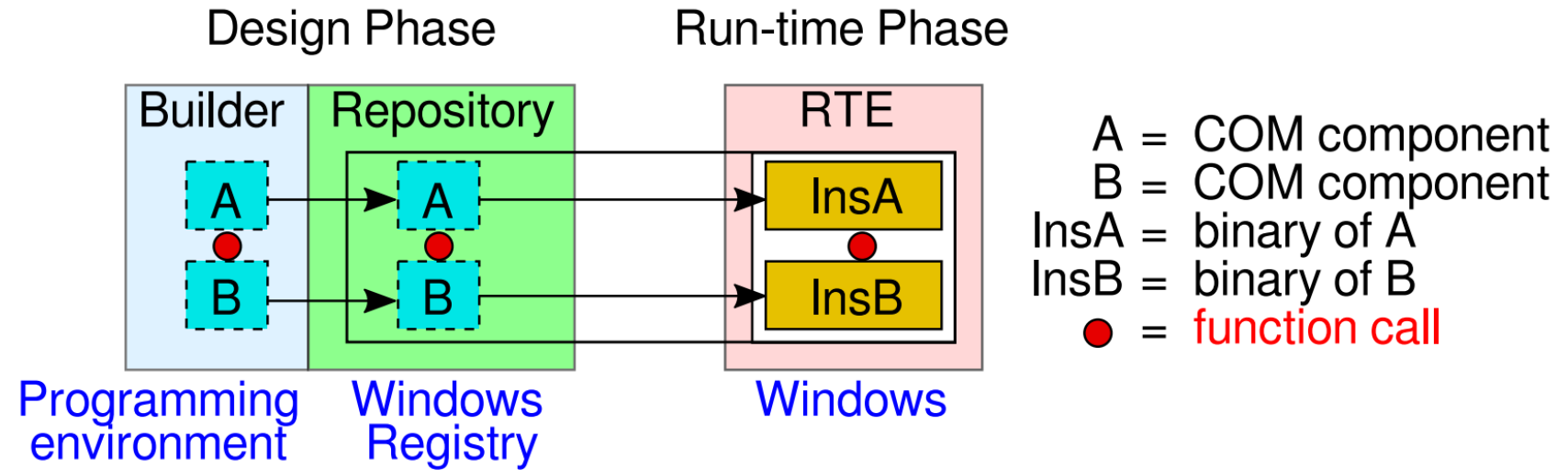


(b) Aggregation

Visibility Compositions:

- **containment** and **aggregation** is possible in COM, but it's just for visibility and
- these compositions don't provide connection between them

4.4 COM (Component Object Model)

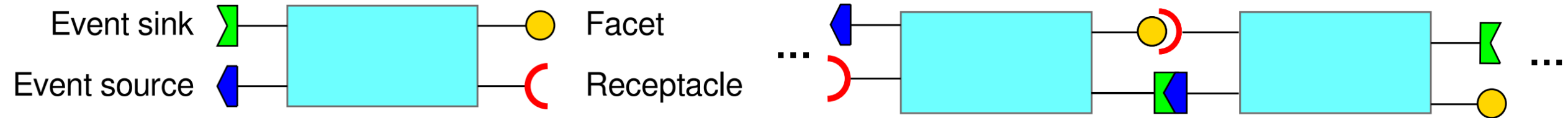


Relation to Idealized

- Design Phase:
 - COM components and composition designed and implemented
 - then deposited in the **Windows Registry**
- Deployment Phase: ***no deployment phase***
- Run-time Phase:
 - client applications call COM components in system via interface pointers

4.5 CCM (COBRA Component Model)

COBRA (Common Object Request Broker Architecture) is designed to facilitate communication between systems deployed on diverse platforms



a **CMM component** -is a COBRA meta-type / special COBRA Object

- this object can be requested in COBRA environment via **IDL (Interface definition language)**
- it is hosted in a **CCM container** on CCM platform such as OpenCCM
- looks like an architectural unit with for types of ports
 - operation interfaces - **facets** => **receptacles**
 - event publish & listen - **event source** => **event sink**

(this composition is specified by **Component Assembly Descriptor- XML**)

4.5 CCM (COBRA Component Model)

Relation to Idealized

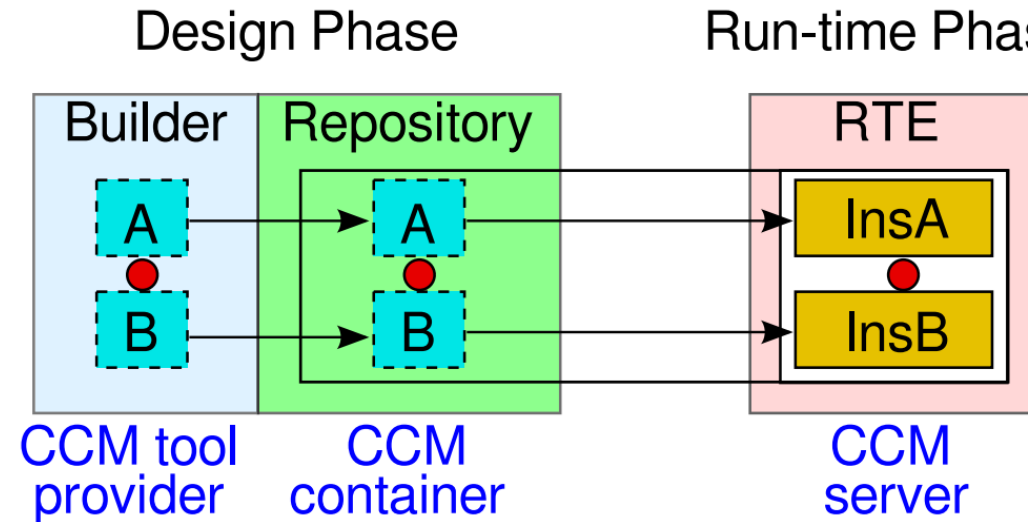
➤ Design Phase

- components are designed in tools like **OpenCCM**
- then deposited into a **CCM container** which is hosted in **CCM platform**

➤ Deployment Phase: no deployment phase

➤ Run-time Phase:

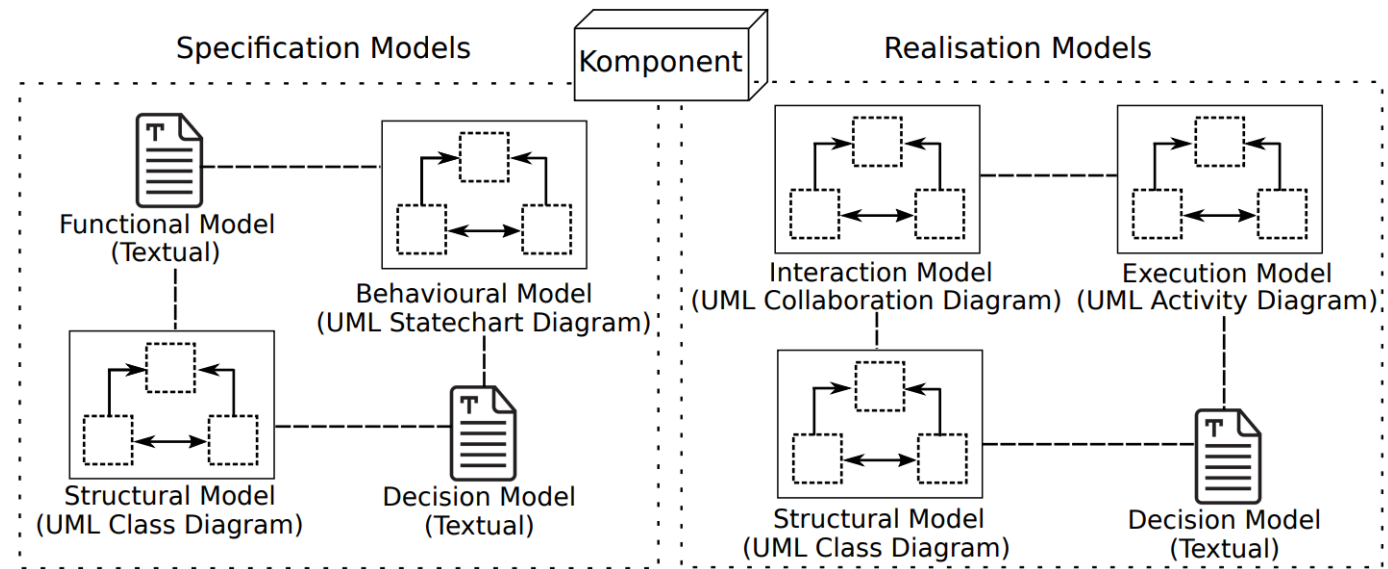
- implementation (assembly file) within CCM Container is pushed to CCM server which deploys and links its instances
- client applications execute these instances by invocation



4.6. KorbA

KorbA is CM for modeling a family of related systems

A KorbA component is defined in UML 1.x not UML 2.x (textually and by UML models) by *specification* and *realization*



- ***Komponent specification*** - models
 - structural - class Diagram
 - behavioral - state chart Diagram
 - functional - textual (words describing expected behavior)
 - decision - textual (conditions to select system from related)

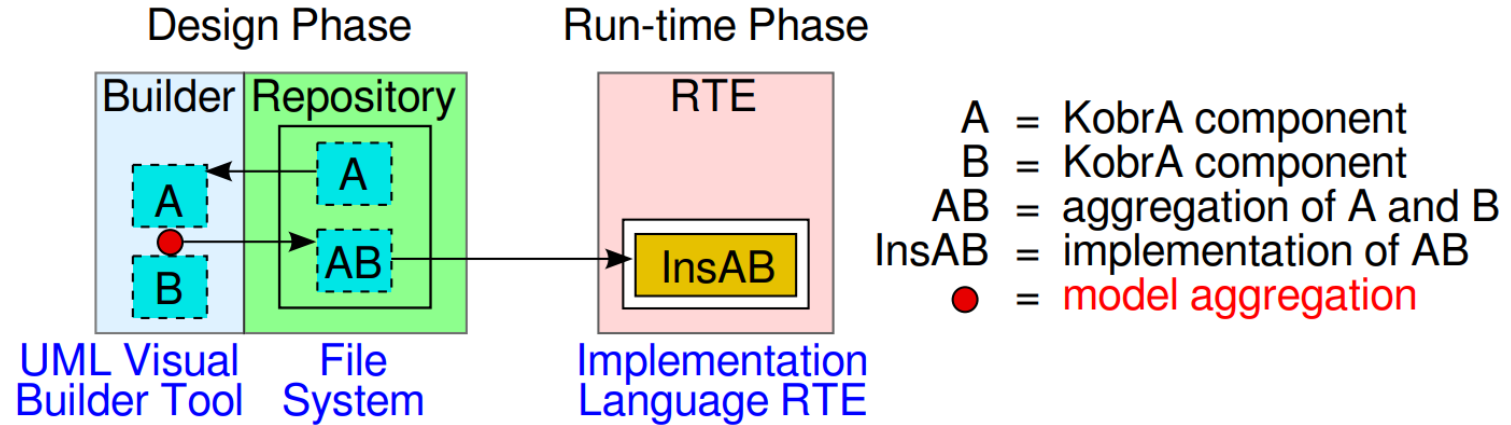
- ***Komponent realization*** - private designs
 - interaction model - (UML collaboration diagram)
 - structural model - (UML class diagram)
 - execution model - (UML activity diagram)
 - decision model - textual (build description)

4.6 KorbA

when implementing UML - KorbA components become Objects in Object oriented Programming language composed by direct method call

Relation to Idealized

- Design Phase:
 - components defined using UML visual builder, stored in builder file
 - models then aggregated into a model for the system according to **decision model**, composed by **method call**
 - this model is implemented in an Object oriented Programming language
- Deployment Phase: there is no deployment phase
- Run-time Phase: executed in the languages runtime environment



Thank You!